UNIVERSITÉ ABDELMALEK ESSAADI
Ecole Nationale des Sciences Appliquées d'Al Hoceima
Département Mathématique et Informatique
Module : J2EE / Java EE
Filière : TDIA 2

Compte Rendu

Application Web Java : Authentification avec Servlets, Sessions et Filters

TP N°3

Encadré par : Pr. Mohamed Cherradi
Réalisé par : Essouiri Aya

Année universitaire : 2025 / 2026

Table des matières

1 Introduction **2**

2 Objectif du TP **2**

3 Technologies utilisées **2**

4 Architecture du projet **2**

5 Modèle : User **3**

6 DAO : UserDao **4**

7 Servlets **5**

7.1 SignUpServlet 5

7.2 LoginServlet 6

7.3 LogoutServlet 7

7.4 HomeServlet 7

8 Gestion des sessions **8**

9 Filter : AuthFilter **8**

10 Interfaces utilisateur **9**

11 Conclusion **11**

1 Introduction

Dans ce TP, nous avons développé une application web basée sur la technologie **Java EE** en utilisant les composants suivants :

— Servlets
— JSP (Java Server Pages)
— Sessions HTTP
— Filters

L'objectif principal de cette application est de mettre en place un système d'authentification permettant aux utilisateurs :

— de créer un compte (Sign Up)
— de se connecter à l'application (Login)
— d'accéder à une page sécurisée
— de se déconnecter (Logout)

Les sessions permettent de maintenir l'état de connexion de l'utilisateur, tandis que les filtres assurent la protection des ressources sensibles de l'application.

2 Objectif du TP

Ce TP vise à comprendre les concepts fondamentaux du développement d'applications web avec Java EE :

— le fonctionnement des Servlets
— la gestion des sessions utilisateurs
— l'utilisation des filtres (Filters)
— l'interaction entre JSP et Servlets

3 Technologies utilisées

Les technologies utilisées dans ce projet sont les suivantes :

— Java EE (Servlet API)
— JSP (Java Server Pages)
— Apache Tomcat 9
— HTML / CSS
— Bootstrap
— Eclipse IDE

4 Architecture du projet

La Figure 1 présente l'architecture générale de l'application.

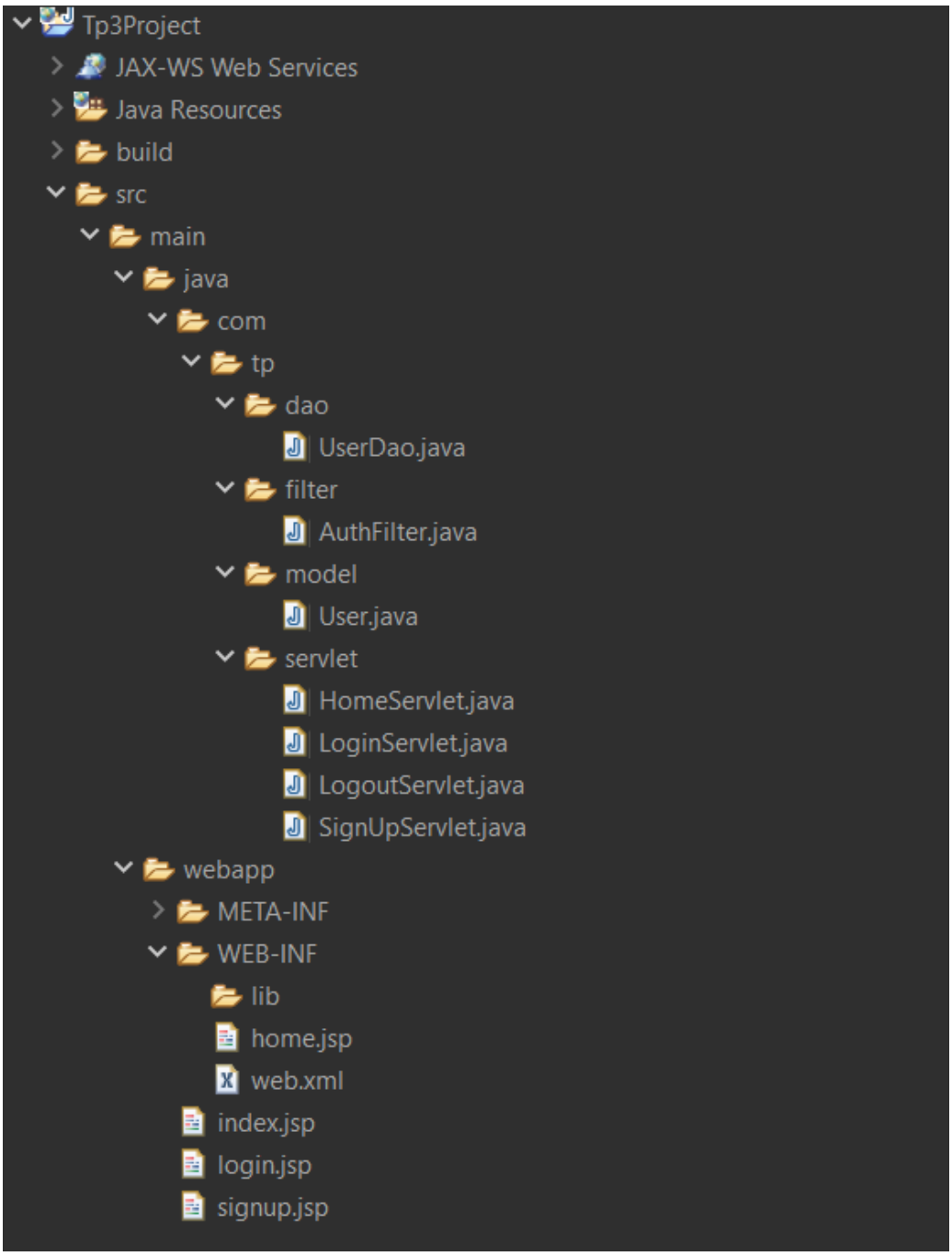

FIGURE 1 – Architecture générale de l'application web Java

5 Modèle : User

La classe **User** représente l'entité principale du système.

Elle contient les informations nécessaires à l'identification d'un utilisateur :

— Nom complet

— Adresse email

— Mot de passe

```java
package com.tp.model;

public class User {
    private String name;
    private String email;
    private String password;

    public User(String name, String email, String password) {
        this.name = name;
        this.email = email;
        this.password = password;
    }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }

}
```

FIGURE 2 – Implémentation de la classe User

6 DAO : UserDao

La classe **UserDao** assure la gestion des utilisateurs dans l'application.

Pour simplifier l'implémentation, les utilisateurs sont stockés dans une structure de données de type **HashMap**.

Les principales méthodes de cette classe sont :

— addUser() : ajouter un utilisateur

— getUserByEmail() : récupérer un utilisateur

— validate() : vérifier les informations de connexion

```java
package com.tp.dao;

import com.tp.model.User;
import java.util.HashMap;
import java.util.Map;

public class UserDao {

    private static Map<String, User> users = new HashMap<>();

    public boolean addUser(User user) {
        if (users.containsKey(user.getEmail())) {
            return false;
        }

        users.put(user.getEmail(), user);
        return true;
    }

    public User getUserByEmail(String email) {
        return users.get(email);
    }

    public boolean validate(String email, String password) {

        User user = users.get(email);

        if (user == null) {
            return false;
        }

        return user.getPassword().equals(password);
    }
}
```

FIGURE 3 – Implémentation de la classe UserDao

7 Servlets

7.1 SignUpServlet

Cette servlet permet de gérer le processus d'inscription des utilisateurs.

Elle effectue les actions suivantes :

— récupération des données du formulaire

— vérification des mots de passe

— enregistrement de l'utilisateur

— redirection vers la page de connexion

```java
public class SignUpServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    private UserDao userDao = new UserDao();

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String password = request.getParameter("password");
        String confirmPassword = request.getParameter("confirmPassword");

        if(!password.equals(confirmPassword)) {
            request.setAttribute("error", "Les mots de passe ne correspondent pas");
            request.getRequestDispatcher("signup.jsp").forward(request, response);
            return;
        }

        User user = new User(name, email, password);
        if(!userDao.addUser(user)) {
            request.setAttribute("error", "Email déjà utilisé");
            request.getRequestDispatcher("signup.jsp").forward(request, response);
            return;
        }

        response.sendRedirect("login.jsp");
    }
}
```

FIGURE 4 – Implémentation de la servlet SignUpServlet

7.2 LoginServlet

La servlet LoginServlet permet d'authentifier les utilisateurs et de créer une session lorsqu'un utilisateur se connecte.

```java
public class LoginServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    private UserDao userDao = new UserDao();

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        String email = request.getParameter("email");
        String password = request.getParameter("password");

        if(userDao.validate(email, password)) {
            HttpSession session = request.getSession();
            session.setAttribute("user", userDao.getUserByEmail(email));
            response.sendRedirect("home");
        } else {
            request.setAttribute("error", "Email ou mot de passe incorrect");
            request.getRequestDispatcher("login.jsp").forward(request, response);
        }
    }
}
```

FIGURE 5 – Implémentation de la servlet LoginServlet

7.3 LogoutServlet

Cette servlet permet de gérer la déconnexion de l'utilisateur en invalidant la session active.

```java
public class LogoutServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        HttpSession session = request.getSession(false);
        if(session != null) {
            session.invalidate();
        }
        response.sendRedirect("login.jsp");
    }
}
```

FIGURE 6 – Implémentation de la servlet LogoutServlet

7.4 HomeServlet

La servlet HomeServlet permet d'afficher la page principale de l'application après authentification.

```java
public class HomeServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        request.getRequestDispatcher("WEB-INF/home.jsp").forward(request, response);
    }
}
```

FIGURE 7 – Implémentation de la servlet HomeServlet

8 Gestion des sessions

Les sessions HTTP permettent de conserver les informations de l'utilisateur pendant sa navigation dans l'application.

Lorsqu'un utilisateur se connecte avec succès, une session est créée afin de stocker ses informations.

9 Filter : AuthFilter

Le filtre AuthFilter permet de sécuriser l'accès à certaines ressources de l'application.

Il vérifie si l'utilisateur possède une session active :

— si la session est inexistante : redirection vers la page login

— sinon : accès autorisé à la page demandée

```java
public class AuthFilter implements Filter {

    public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
            throws IOException, ServletException {

        HttpServletRequest request = (HttpServletRequest) req;
        HttpServletResponse response = (HttpServletResponse) res;

        HttpSession session = request.getSession(false);
        if(session != null && session.getAttribute("user") != null) {
            chain.doFilter(req, res);
        } else {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
        }
    }
}
```

FIGURE 8 – Implémentation du filtre AuthFilter

10 Interfaces utilisateur

Les interfaces de l'application ont été développées en utilisant JSP et Bootstrap afin d'améliorer l'expérience utilisateur.

Les principales pages développées sont :

— page d'accueil
— page de connexion
— page d'inscription
— page utilisateur

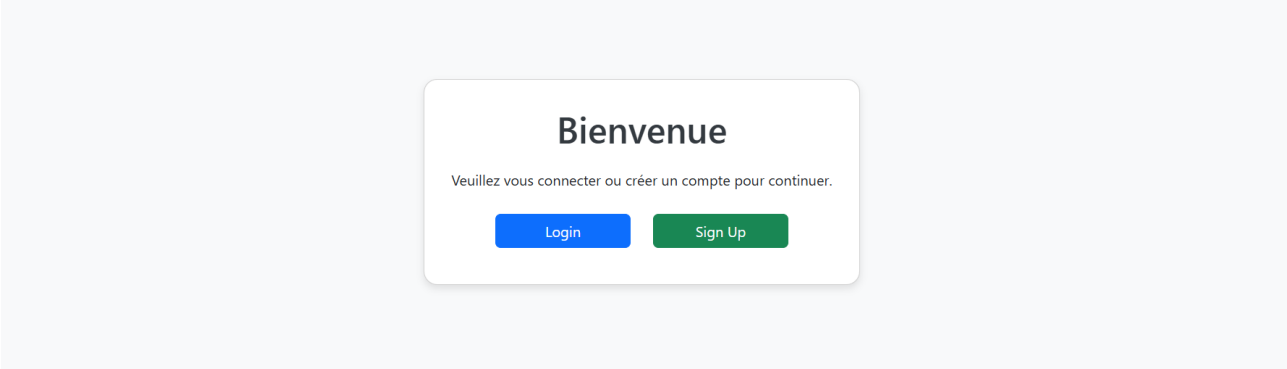

FIGURE 9 – Interface principale - page d'accueil

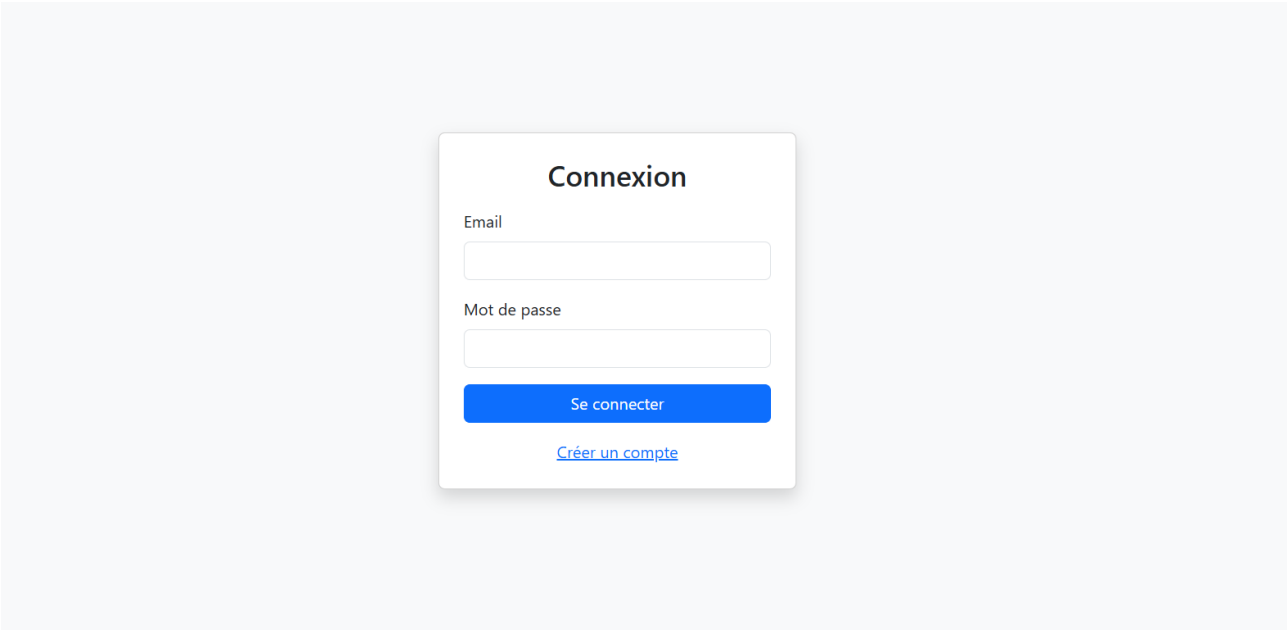

FIGURE 10 – Formulaire de connexion (login)

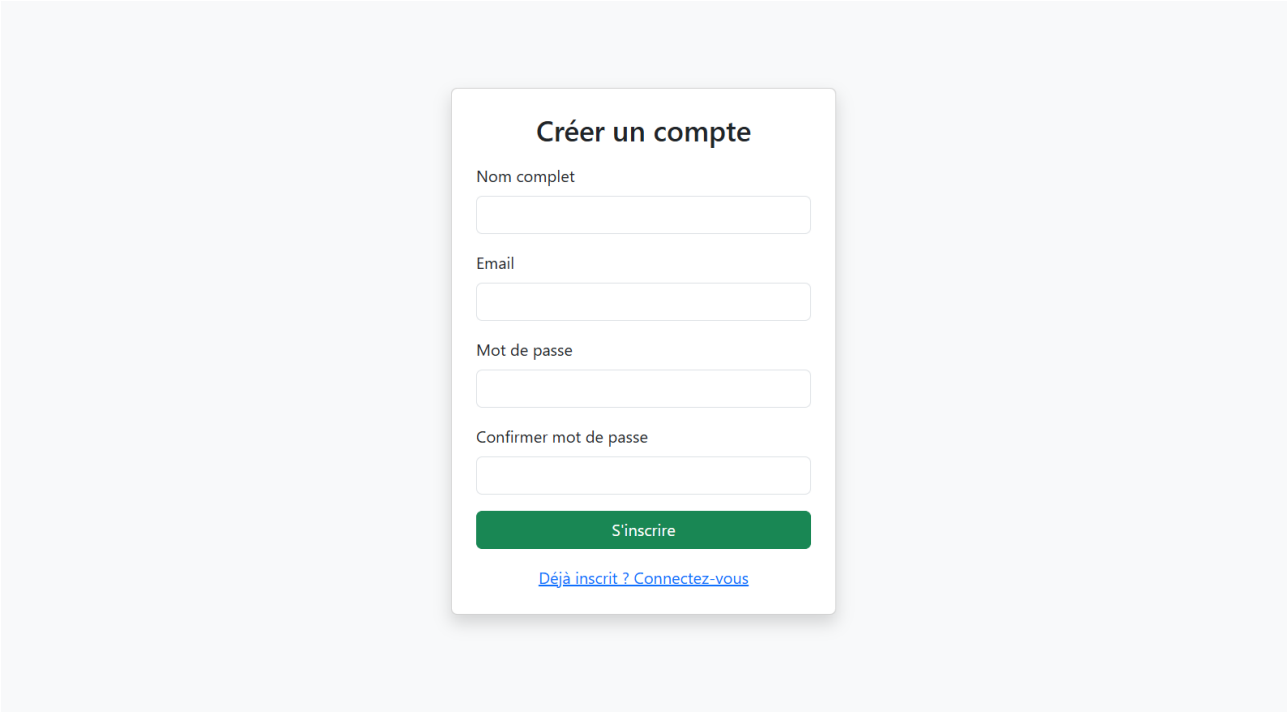

FIGURE 11 – Formulaire d'inscription (signup)

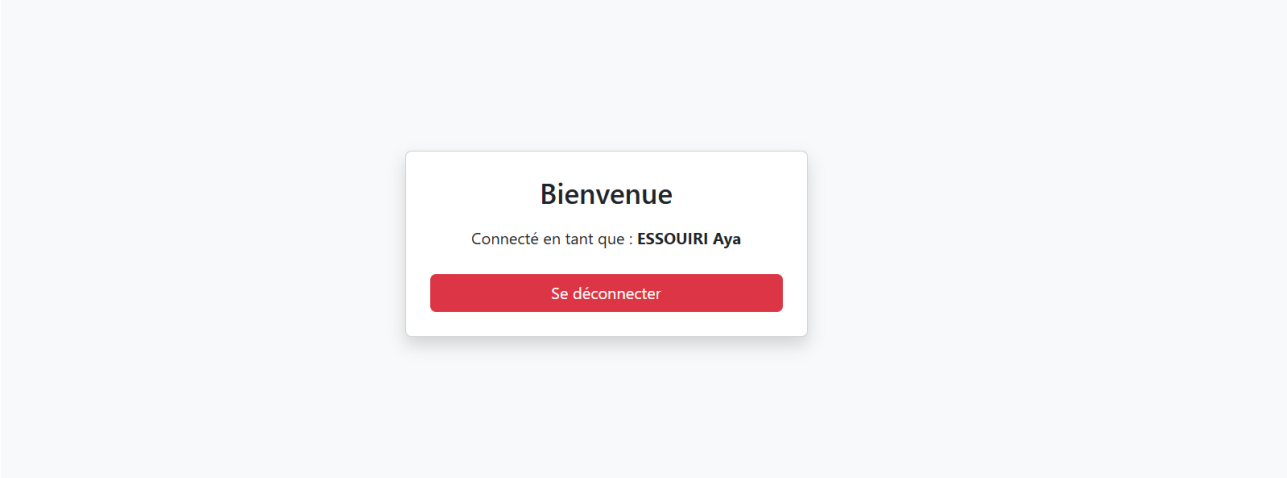

FIGURE 12 – Page d'accueil après authentification (home)

11 Conclusion

Ce TP nous a permis de comprendre les concepts fondamentaux du développement web avec Java EE.

Nous avons appris à utiliser :

— les Servlets

— les JSP

— les sessions HTTP

— les filtres pour sécuriser les ressources

Ces notions constituent la base du développement d'applications web dynamiques en Java.